Programming Languages and Computation

Week 7: Operational Semantics

1 Operational Semantics

This section is about the small-step operational semantics of While programs as given by the relation $\to \subseteq \mathcal{C} \times \mathcal{C}$ where the configurations are either a statement and a state or a state $\mathcal{C} = (\mathsf{S} \times \mathsf{State}) \cup \mathsf{State}$, which is defined by these following rules:

$$\frac{}{\langle \mathsf{skip}, \sigma \rangle \to \sigma} \qquad \frac{}{\langle x \leftarrow e, \sigma \rangle \to \sigma[x \mapsto [\![e]\!]_{\mathcal{A}}(\sigma)]}$$

$$\frac{\langle S_1, \sigma_1 \rangle \to \sigma_2}{\langle S_1;\ S_2, \sigma_1 \rangle \to \langle S_2, \sigma_2 \rangle} \qquad \frac{\langle S_1, \sigma_1 \rangle \to \langle S_1', \sigma_2 \rangle}{\langle S_1;\ S_2, \sigma_1 \rangle \to \langle S_1';\ S_2, \sigma_2 \rangle}$$

$$\frac{}{\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma \rangle \to \langle S_1, \sigma \rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \top$$

$$\frac{}{\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma \rangle \to \langle S_2, \sigma \rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \bot$$

$$\frac{}{\langle \mathsf{while}\ e\ \{S\}, \sigma \rangle \to \sigma}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \bot$$

$$\frac{}{\langle \mathsf{while}\ e\ \{S\}, \sigma \rangle \to \langle S;\ \mathsf{while}\ e\ \{S\}, \sigma \rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \top$$

Figure 1: Operational semantics for While.

* 1. Calculate the terminal state and write down a trace for the statement $x \leftarrow 1; x \leftarrow 2; x \leftarrow 3$ with the initial state $[]$ using the rules in Figure 1. Remember, variables are assigned 0 by default.

Solution

The terminal state is $[x \mapsto 3]$ and the corresponding trace is:

$$\begin{aligned} &\langle x \leftarrow 1;\ x \leftarrow 2;\ x \leftarrow 3, [] \rangle \\ &\quad \to \langle x \leftarrow 2;\ x \leftarrow 3, [x \mapsto 1] \rangle \\ &\quad \to \langle x \leftarrow 3, [x \mapsto 2] \rangle \\ &\quad \to [x \mapsto 3] \end{aligned}$$

* 2. Calculate the terminal state and write down a trace for the statement $x \leftarrow 1; x \leftarrow x * 2; x \leftarrow x + y$ with the initial state $[x \mapsto 2, y \mapsto 2]$ using the rules in Figure 1.

Solution

The terminal state is $[x \mapsto 4, y \mapsto 2]$ and the corresponding trace is:

$$\begin{aligned}
&\langle \{x \leftarrow 1; x \leftarrow x * 2\}; x \leftarrow x + y, [x \mapsto 2, y \mapsto 2]\rangle \\
&\quad \rightarrow \langle x \leftarrow x * 2; x \leftarrow x + y, [x \mapsto 1, y \mapsto 2]\rangle \\
&\quad \rightarrow \langle x \leftarrow x + y, [x \mapsto 2, y \mapsto 2]\rangle \\
&\quad \rightarrow [x \mapsto 4, y \mapsto 2]
\end{aligned}$$

* 3. Find a state σ such that $\langle x \leftarrow 1; y \leftarrow x * 2, [\,]\rangle \rightarrow^* \sigma$. You should provide the corresponding trace.

Solution

The state $[x \mapsto 1, y \mapsto 2]$ satisfies the requirement with the associated trace:

$$\begin{aligned}
&\langle x \leftarrow 1; y \leftarrow x * 2, [\,]\rangle \\
&\quad \rightarrow \langle y \leftarrow x * 2, [x \mapsto 1]\rangle \\
&\quad \rightarrow [x \mapsto 1, y \mapsto 2]
\end{aligned}$$

* 4. Compute the final state for the program if $x \leq y$ $\{x \leftarrow y\}$ else $\{y \leftarrow x\}$ when executed in each of the following states:

- $[\,]$
- $[x \mapsto 2, y \mapsto 3]$
- $[x \mapsto 4, y \mapsto 2]$

Solution

- $[\,]$
- $[x \mapsto 3, y \mapsto 3]$
- $[x \mapsto 4, y \mapsto 4]$

* 5. Find a state $\sigma \in \text{State}$ such that $\text{while } !(x \leq -1)\ \{x \leftarrow x + d\}, [d \mapsto -1] \rightarrow^* \sigma$. You should provide the corresponding trace.

Solution

The state $[d \mapsto -1, x \mapsto -1]$ will satisfy the requirements. The associated trace is:

$$\begin{aligned}
&\langle \text{while } !(x \leq -1)\ \{x \leftarrow x + d\}, [d \mapsto -1]\rangle \\
&\quad \rightarrow \langle x \leftarrow x + d;\ \text{while } !(x \leq -1)\ \{x \leftarrow x + d\}, [d \mapsto -1]\rangle \\
&\quad \rightarrow \langle \text{while } !(x \leq -1)\ \{x \leftarrow x + d\}, [d \mapsto -1, x \mapsto -1]\rangle \\
&\quad \rightarrow [d \mapsto -1, x \mapsto -1]
\end{aligned}$$

* 6. Find a state $\sigma \in \mathsf{State}$ such that $\langle x \leftarrow 2; y \leftarrow x * y, \sigma\rangle \rightarrow^* [x \mapsto 2, y \mapsto 4]$. You should provide the corresponding trace.

Solution

The state $[y \mapsto 2]$ will satisfy the requirements. The associated trace is:

$$\begin{aligned}&\langle x \leftarrow 2; y \leftarrow x * y, [y \mapsto 2]\rangle\\ &\quad\rightarrow \langle y \leftarrow x * y, [x \mapsto 2, y \mapsto 2]\rangle\\ &\quad\rightarrow [x \mapsto 2, y \mapsto 4]\end{aligned}$$

** 7. Suppose $e \in \mathcal{B}$ is a Boolean expression that is semantically equivalent to false. Prove that $\langle \mathsf{while}\ e\ \{S\}, \sigma\rangle \rightarrow \sigma$ for any state $\sigma \in \mathsf{State}$.

Solution

As $e \in \mathcal{B}$ is semantically equivalent to false, we have that $[\![e]\!]_{\mathcal{B}}(\sigma) = \bot$ for any state $\sigma \in \mathsf{State}$. Therefore, we have that $\langle \mathsf{while}\ e\ \{S\}, \sigma\rangle \rightarrow \sigma$ for any state $\sigma \in \mathsf{State}$ as required.

** 8. Suppose $S_1, S_2 \in \mathcal{S}$ are two statements such that $\langle S_1, \sigma\rangle \rightarrow^* \sigma'$ and $\langle S_2, \sigma\rangle \rightarrow^* \sigma'$ for some states $\sigma, \sigma' \in \mathsf{State}$. Prove that $\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma\rangle \rightarrow^* \sigma'$ for any Boolean expression $e \in \mathcal{B}$.

Solution

Let $e \in \mathcal{B}$ be some Boolean expression and $\sigma, \sigma' \in \mathsf{State}$ be two states. Let us consider two cases:

- Suppose $[\![e]\!]_{\mathcal{B}}(\sigma)$ is true. Then we have the following trace:

$$\begin{aligned}&\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma\rangle\\ &\quad\rightarrow \langle S_1, \sigma\rangle\\ &\quad\rightarrow^* \sigma'\end{aligned}$$

- Suppose, otherwise, that $[\![e]\!]_{\mathcal{B}}(\sigma)$ is false. Then equally we have the following trace of the form:

$$\begin{aligned}&\langle \mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}, \sigma\rangle\\ &\quad\rightarrow \langle S_2, \sigma\rangle\\ &\quad\rightarrow^* \sigma'\end{aligned}$$

** 9. Prove that, for any statement $S \in \mathcal{S}$ and any Boolean expression $e \in \mathcal{B}$, the statement $\mathsf{if}\ e\ \{S\}\ \mathsf{else}\ \{S\}$ is semantically equivalent to S.

Solution

Suppose that $\langle S, \sigma\rangle \rightarrow^* \sigma'$. Either $[\![e]\!]_{\mathcal{B}}(\sigma) = \top$ and $\langle \mathsf{if}\ e\ \{S\}\ \mathsf{else}\ \{S\}, \sigma\rangle \rightarrow \langle S, \sigma\rangle$ or $[\![e]\!]_{\mathcal{B}}(\sigma) = \bot$ and $\langle \mathsf{if}\ e\ \{S\}\ \mathsf{else}\ \{S\}, \sigma\rangle \rightarrow \langle S, \sigma\rangle$. Therefore, in either case, $\langle \mathsf{if}\ e\ \{S\}\ \mathsf{else}\ \{S\}, \sigma\rangle \rightarrow^* \sigma$.

Conversely, if $\langle \mathsf{if}\ e\ \{S\}\ \mathsf{else}\ \{S\}, \sigma\rangle \rightarrow^* \sigma'$, then either $[\![e]\!]_{\mathcal{B}}(\sigma) = \top$ and $\langle S, \sigma\rangle \rightarrow^* \sigma'$ or likewise for $[\![e]\!]_{\mathcal{B}}(\sigma) = \bot$. In either case, $\langle S, \sigma\rangle \rightarrow \sigma'$.

** 10. Prove that, for any statements $S_1, S_2 \in \mathcal{S}$ and any Boolean expression $e \in \mathcal{B}$, the statement $\mathsf{if}\ e\ \{S_1\}\ \mathsf{else}\ \{S_2\}$ is semantically equivalent to $\mathsf{if}\ !e\ \{S_2\}\ \mathsf{else}\ \{S_1\}$.

Solution

Suppose that if e $\{S_1\}$ else $\{S_2\} \to^* \sigma'$. Then either $[\![e]\!]_{\mathcal{B}}(\sigma) = \top$ and $\langle S_1, \sigma\rangle \to^* \sigma'$ or $[\![e]\!]_{\mathcal{B}}(\sigma) = \bot$ and $\langle S_2, \sigma\rangle \to^* \sigma'$. In the first case, we have that $[\![!e]\!]_{\mathcal{B}}(\sigma) = \bot$ and hence $\langle$if $!e$ $\{S_2\}$ else $\{S_1\}, \sigma\rangle \to \langle S_1, \sigma\rangle \to^* \sigma'$ as required. The second case is analogous.

The same argument applies in the converse direction, except using the fact that $[\![!e]\!]_{\mathcal{B}}(\sigma) = \top$ implies $[\![!e]\!]_{\mathcal{B}}(\sigma) = \bot$.

** 11. Suppose we introduce a new language construct "do S while e" where $S \in \mathcal{S}$ is a statement and $e \in \mathcal{B}$ is a Boolean expression. The operational semantics for this construct is given by the following inference rules:

$$\frac{}{\langle \text{do } \{S\} \text{ while } e, \sigma\rangle \to \langle S; \text{ if } e \text{ } \{\text{do } \{S\} \text{ while } e\} \text{ else } \{\text{skip}\}, \sigma\rangle}$$

(a) Find a state $\sigma \in \text{State}$ such that $\langle \text{do } \{x \leftarrow x + 1\} \text{ while } x \leq 1, [\,]\rangle \to^* \sigma$ and give the associated trace.

(b) For a given statement $S \in \mathcal{S}$ and a Boolean expression $e \in \mathcal{B}$ find a While program that is equivalent to the program do $\{S\}$ while e but does not use the new construct. That is, find a statement $S' \in \mathcal{S}$ such that:

$$\langle S', \sigma\rangle \to^* \sigma' \Leftrightarrow \langle \text{do } \{S\} \text{ while } e, \sigma\rangle \to^* \sigma'$$

You do not need to prove that your answer is correct but should provide a trace for $\langle S', [\,]\rangle \to^* \sigma$ where S is given to be the statement $x \leftarrow x + 1$, where e is given to be the expression $x \leq 1$, and where σ is the state from part (a).

Solution

(a) The state is $[x \mapsto 2]$ and the trace is:

$$\begin{aligned}
&\langle \text{do } x \leftarrow x + 1 \text{ while } x \leq 1, [\,]\rangle \\
&\quad \to \langle x \leftarrow x + 1; \text{ if } x \leq \ \{\text{do } x \leftarrow x + 1 \text{ while } x \leq 1\} \text{ else skip}, [\,]\rangle \\
&\quad \to \langle \text{if } x \leq 1\{\text{do } x \leftarrow x + 1 \text{ while } x \leq 1\} \text{ else skip}, [x \mapsto 1]\rangle \\
&\quad \to \langle \text{do } x \leftarrow x + 1 \text{ while } x \leq 1, [x \mapsto 1]\rangle \\
&\quad \to \langle x \leftarrow x + 1; \text{ if } x \leq \ \{\text{do } x \leftarrow x + 1 \text{ while } x \leq 1\} \text{ else skip}, [x \mapsto 1]\rangle \\
&\quad \to \langle \text{if } x \leq 1\{\text{do } x \leftarrow x + 1 \text{ while } x \leq 1\} \text{ else skip}, [x \mapsto 2]\rangle \\
&\quad \to \langle \text{skip}, [x \mapsto 2]\rangle \\
&\quad \to [x \mapsto 2]
\end{aligned}$$

(b) The statement S; while e $\{S\}$ is equivalent.

$$\begin{aligned}
&\langle x \leftarrow x + 1; \text{ while } x \leq 1 \ \{x \leftarrow x + 1\}, [\,]\rangle \\
&\quad \to \langle \text{while } x \leq 1 \ \{x \leftarrow x + 1\}, [x \mapsto 1]\rangle \\
&\quad \to \langle x \leftarrow x + 1; \text{ while } x \leq 1 \ \{x \leftarrow x + 1\}, [x \mapsto 1]\rangle \\
&\quad \to \langle \text{while } x \leq 1 \ \{x \leftarrow x + 1\}, [x \mapsto 2]\rangle \\
&\quad \to [x \mapsto 2]
\end{aligned}$$

** 12. Suppose we introduce a new language construct for x do S where $S \in \mathcal{S}$ is a statement and $x \in \mathsf{Var}$ is a variable The operational semantics for this construct is given by the following inference rules:

$$\frac{}{\langle \text{for } x \text{ do } S, \sigma\rangle \to \sigma}\ \sigma(x) \leq 0 \qquad \frac{}{\langle \text{for } x \text{ do } S, \sigma_1\rangle \to \langle S;\ \text{for } x \text{ do } S, \sigma[x \mapsto \sigma(x)-1]\rangle}\ \sigma(x) > 0$$

(a) Find a state $\sigma \in \mathsf{State}$ such that $\langle \text{for } x \text{ do } y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 3]\rangle \to^* \sigma$ and give the associated trace.

(b) For a given statement $S \in \mathcal{S}$ and a variable $x \in \mathsf{Var}$ find a While program that is equivalent to the program for x do S but does not use the new construct. That is, find a statement $S' \in \mathcal{S}$ such that:

$$\langle S', \sigma\rangle \to^* \sigma' \iff \langle \text{for } x \text{ do } S, \sigma\rangle \to^* \sigma'$$

You do not need to prove that your answer is correct but should provide a trace for $\langle S', [x \mapsto 3]\rangle \to^* \sigma$ where S is given to be the statement $y \leftarrow y + x;\ x \leftarrow x - 2$ and σ is the state from part (a).

Solution

(a)

$$\begin{aligned}
&\langle \text{for } x \text{ do } y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 3]\rangle \\
&\quad \to \langle y \leftarrow y + x;\ x \leftarrow x - 2;\ \text{for } x \text{ do } y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 2]\rangle \\
&\quad \to \langle x \leftarrow x - 2;\ \text{for } x \text{ do } y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 2, y \mapsto 2]\rangle \\
&\quad \to \langle \text{for } x \text{ do } y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 0, y \mapsto 2]\rangle \\
&\quad \to [x \mapsto 0, y \mapsto 2]
\end{aligned}$$

(b) The statement while $!(x \leq 0)$ $\{x \leftarrow x - 1;\ S\}$ is equivalent.

$$\begin{aligned}
&\langle \text{while } !(x \leq 0) \text{ do } x \leftarrow x - 1;\ y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 3]\rangle \\
&\quad \to \langle x \leftarrow x - 1;\ y \leftarrow y + x;\ x \leftarrow x - 2;\ \text{while } !(x \leq 0) \text{ do } x \leftarrow x - 1;\ y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 3]\rangle \\
&\quad \to^* \langle \text{while } !(x \leq 0) \text{ do } x \leftarrow x - 1;\ y \leftarrow y + x;\ x \leftarrow x - 2, [x \mapsto 0, y \mapsto 2]\rangle \\
&\quad \to [x \mapsto 0, y \mapsto 2]
\end{aligned}$$

*** 13. Show that, if $y \notin \mathsf{FV}(e_1)$ and $x \notin \mathsf{FV}(e_2)$ where x and y are distinct variables, then the statements $x \leftarrow e_1;\ y \leftarrow e_2$ and $y \leftarrow e_2;\ x \leftarrow e_1$ equivalent in the sense that $\langle x \leftarrow e_1;\ y \leftarrow e_2, \sigma\rangle \to^* \sigma'$ if and only if $\langle y \leftarrow e_2;\ x \leftarrow e_1, \sigma\rangle \to^* \sigma'$. You may use results from the previous worksheet.

Solution

This is most straightforward proven from the fact that $[\![e_1]\!]_{\mathcal{A}}(\sigma) = [\![e_1]\!]_{\mathcal{A}}(\sigma')$ if $\sigma(x) = \sigma'(x)$ for all $x \in \mathsf{FV}(e_1)$, and likewise for e_2.

Note that $\langle x \leftarrow e_1;\ y \leftarrow e_2, \sigma_0\rangle \to^* \sigma_2$ just if $\sigma_1 = \sigma_0[x \mapsto [\![e_1]\!]_{\mathcal{A}}(\sigma_0)]$ and $\sigma_2 = \sigma_1[y \mapsto [\![e_2]\!]_{\mathcal{A}}(\sigma_1)]$. And, likewise $\langle y \leftarrow e_2;\ x \leftarrow e_1, \sigma_0\rangle \to^* \sigma_2'$ just if $\sigma_1' = \sigma_0[y \mapsto [\![e_2]\!]_{\mathcal{A}}(\sigma_0)]$ and $\sigma_2' = \sigma_1'[x \mapsto [\![e_1]\!]_{\mathcal{A}}(\sigma_1')]$.

To show that σ_2 is equal to σ_2', we must show that $[\![e_1]\!]_{\mathcal{A}}(\sigma_0) = [\![e_1]\!]_{\mathcal{A}}(\sigma_1')$ and that $[\![e_2]\!]_{\mathcal{A}}(\sigma_1) = [\![e_2]\!]_{\mathcal{A}}(\sigma_0)$. Using the previous result, and the fact that $\sigma_0(z) = \sigma_1'(z)$ for all $z \neq y$, we have that $[\![e_1]\!]_{\mathcal{A}}(\sigma_0) = [\![e_1]\!]_{\mathcal{A}}(\sigma_1')$ as $y \notin \mathsf{FV}(e_1)$. And likewise $[\![e_2]\!]_{\mathcal{A}}(\sigma_1) = [\![e_2]\!]_{\mathcal{A}}(\sigma_0)$ follows from $x \notin \mathsf{FV}(e_2)$. Therefore, $\sigma_2 = \sigma_2'$ as required.

*** 14. The set of variables *modified* by a statement is defined by recursion as follows:

$$\begin{aligned}
&\mathsf{mod}(\mathsf{skip}) = \emptyset \\
&\mathsf{mod}(x \leftarrow e) = \{x\} \\
&\mathsf{mod}(S_1;\ S_2) = \mathsf{mod}(S_1) \cup \mathsf{mod}(S_2) \\
&\mathsf{mod}(\mathsf{if}\ e\ \{S_1\}\mathsf{else}\ \{S_2\}) = \mathsf{mod}(S_1) \cup \mathsf{mod}(S_2) \\
&\mathsf{mod}(\mathsf{while}\ e\ \{S\}) = \mathsf{mod}(S)
\end{aligned}$$

(a) Prove that if $\langle S_1, \sigma_1\rangle \to \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$.

(b) Prove that if $\langle S_1, \sigma_1\rangle \to \langle S_2, \sigma_2\rangle$ then $\mathsf{mod}(S_2) \subseteq \mathsf{mod}(S_1)$ and $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$. You may use the previous result.

(c) Prove that if $\langle S_1, \sigma_1\rangle \to^* \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$. You may use the previous results, and the fact that $\gamma_1 \to^* \gamma_2$ if, and only if, $\gamma_1 \to^n \gamma_2$ for some $n \geq 0$.

Solution

(a) First, we shall prove that $\langle S_1, \sigma_1\rangle \to \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$. There are only three cases for this:

- If $\langle \mathsf{skip}, \sigma\rangle \to \sigma$ then we immediately have that $\sigma(x) = \sigma(x)$ for all $x \notin \mathsf{mod}(\mathsf{skip})$ as required.
- If $\langle x \leftarrow e, \sigma\rangle \to \sigma[x \mapsto [\![e]\!]_{\mathcal{A}}(\sigma)]$ then we have that $\sigma(y) = \sigma[x \mapsto [\![e]\!]_{\mathcal{A}}(\sigma)](y)$ for all $y \notin \mathsf{mod}(x \leftarrow e)$ as $x \in \mathsf{mod}(x \leftarrow e)$.
- If $\langle \mathsf{while}\ e\ \mathsf{do}\ S, \sigma\rangle \to \sigma$ when $[\![e]\!]_{\mathcal{B}}(\sigma) = \bot$, then again we immediately have that $\sigma(x) = \sigma(x)$ for all $x \notin \mathsf{mod}(\mathsf{while}\ e\ \mathsf{do}\ S)$ as required.

(b) To complete the second part of this proof, you need to use induction over the statement. However, it is only relevant in the sequence case(s) — the induction hypothesis is not important for any other case.
We shall prove that if $\langle S_1, \sigma_1\rangle \to \langle S_1, \sigma_2\rangle$ then $\mathsf{mod}(S_2) \subseteq \mathsf{mod}(S_1)$ and $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$ by structural induction over S_1.

- In the case of skip or an assignment, there is nothing to prove as it steps to a terminal configuration.
- For the statement $S_1;\ S_2$, we have $\langle S_1;\ S_2, \sigma_1\rangle \to \langle S_1';\ S_2, \sigma_2\rangle$ when $\langle S_1\, \sigma_1\rangle \to \langle S_1', \sigma_2\rangle$. By induction, we have that $\mathsf{mod}(S_1') \subseteq \mathsf{mod}(S_1)$ and $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$. It then follows that $\mathsf{mod}(S_1';\ S_2) \subseteq \mathsf{mod}(S_1;\ S_2)$ and $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1;\ S_2)$ as $\mathsf{mod}(S_1) \subseteq \mathsf{mod}(S_1;\ S_2)$.
  If, on the other hand, we have $\langle S_1;\ S_2, \sigma_1\rangle \to \langle S_2, \sigma_2\rangle$ when $\langle S_1\, \sigma_1\rangle \to \sigma_2\rangle$, the by the previous result $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$ and thus by extension $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1;\ S_2)$ as $\mathsf{mod}(S_1) \subseteq \mathsf{mod}(S_1;\ S_2)$.
- For the $\mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2$, we either have $\langle \mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2, \sigma\rangle \to \langle S_1, \sigma\rangle$ or $\langle \mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2, \sigma_1\rangle \to \langle S_2, \sigma\rangle$. In either case, we have that $\mathsf{mod}(S_1), \mathsf{S}_2 \subseteq \mathsf{mod}(\mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2)$. Additionally, $\sigma(x) = \sigma(x)$ for all $x \notin \mathsf{mod}(\mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2)$.
- The case of while is analogous to the former.

(c) The final part of this exercise is to prove that $\langle S_1, \sigma_1 \rangle \rightarrow^n \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$. This is done by induction over n, i.e. the length of the trace.

- The base case is absurd as to reach a termination configuration from a non-termination configuration requires at least one step.
- Suppose that $\langle S_1, \sigma_1 \rangle \rightarrow \gamma$ and $\gamma \rightarrow^n \sigma_2$. There are two cases to consider:
  - γ is the terminal configuration σ_2. In which case, by the previous results, $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$ as required.
  - γ is another non-terminal configuration $\langle S_2, \sigma_3 \rangle$. In which case, by the previous results, $\sigma_1(x) = \sigma_3(x)$ for all $x \notin \mathsf{mod}(S_1)$ and $\mathsf{mod}(S_2) \subseteq \mathsf{mod}(S_1)$. By the induction hypothesis, $\sigma_3(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_2)$, and so $\sigma_3(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$ as $\mathsf{mod}(S_2) \subseteq \mathsf{mod}(S_1)$. It follows that $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \mathsf{mod}(S_1)$ as required.